

Project Interim Report
On
AGRI-PRICE PREDICTOR

RU-500-02-00002

Foundations of Machine Learning

SESSION: 2026-27



Submitted By-
NAME – ANSH GIRI GOSWAMI
ERP – RU-25-10231

Master of Computer Application in
Association With Google

Under the Guidance of
Mr. ZAMIN HAMID

SCHOOL OF INFORMATION AND TECHNOLOGY
RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

ABSTRACT

Agro Predictor is a machine learning-based web application developed to forecast short-term market prices for critical agricultural commodities—specifically onions and tomatoes—by analyzing historical market data and external influencing variables. Price volatility remains a pervasive challenge in the agricultural sector, exposing farmers, traders, wholesalers, and consumers to significant financial uncertainty. Due to rapid fluctuations driven by seasonal demand, weather anomalies, and supply chain disruptions, stakeholders frequently struggle to optimize the timing of crop sales, purchases, and storage. Agro Predictor addresses this gap by replacing subjective intuition and informal market rumors with an accessible, data-driven forecasting framework.

At its core, the platform implements a structured machine learning pipeline that transforms raw historical data from CSV format into actionable market intelligence. During the preprocessing phase, the application executes extensive feature engineering to capture temporal and environmental dynamics. This includes extracting time-based indicators (months, weeks, and days of the year), modeling cyclical time features, computing lagged price values, generating rolling averages and momentum indicators, and integrating key weather variables. Because agricultural markets exhibit non-random, path-dependent trends, these engineered features allow the system to map complex relationships and achieve higher predictive accuracy than traditional manual estimation methods.

The predictive engine relies on Random Forest regression models, selected for their capacity to handle non-linear relationships, manage mixed feature types, and mitigate overfitting compared to standard linear regression models. The architecture features an automated training and evaluation utility, enabling continuous model refinement and redeployment as fresh datasets become available. To optimize runtime efficiency, trained models are serialized and persisted in a compact format, ensuring rapid inference without requiring on-the-fly retraining during web application startup. In the event of missing serialized assets, a built-in fallback mechanism automatically rebuilds the models directly from the source data, securing deployment stability.

The application is deployed as a full-stack web system utilizing a lightweight Flask backend and an intuitive, responsive user interface. To maximize practical utility, the user input requirements are kept minimal. Rather than delivering an isolated numeric forecast, the application outputs a comprehensive prediction profile that includes the estimated price trend, projected percentage changes, a confidence-bounded forecast range, and interactive data previews for model transparency. Additionally, the system exposes a set of RESTful API endpoints, allowing the forecasting engine to be integrated programmatically into external agricultural digital tools.

Ultimately, Agro Predictor bridges the gap between data science and practical agricultural planning. By rendering complex predictive analytics into a simple, real-time decision support tool, the project provides small-scale stakeholders with the data visibility needed to minimize post-harvest losses, optimize procurement strategies, and navigate market volatility. This project serves as a robust academic and technical demonstration of how integrated machine learning pipelines, model persistence strategies, and web technologies can be unified to solve domain-specific economic challenges.

Key Technical Stack Highlighted:

- Backend Framework: Flask (Python)
- Predictive Model: Random Forest Regressor
- Feature Engineering: Cyclical time encoding, rolling averages,
- price momentum, lag features, and weather variables.

INTRODUCTION

1. Background and Motivation

Libraries serve as essential hubs for resource sharing, but managing physical book catalogs and member logs presents severe operational challenges when handled manually. Traditionally, institutions relied on paper ledgers or basic spreadsheets to track acquisitions and borrowing transactions. This manual approach introduces data errors, causes fragmentation, and makes checking book availability or tracking overdue assets incredibly slow. Transitioning to a dedicated digital application is essential to streamline these daily workflows, protect data integrity, and provide immediate inventory updates.

2. Problem Statement

Manual library administration creates operational bottlenecks for both librarians and members. Without centralized tracking, it is difficult to instantly determine which books are on the shelves, who holds an overdue title, or when an item will be returned. From a software perspective, many basic applications lack a scalable architecture, resulting in unorganized code that is hard to maintain or expand. To fix this, there is a clear need for a clean, automated management tool built on modern engineering practices that simplifies data handling and solves daily workflow challenges.

3. Project Objectives

The primary goal of this project is to eliminate administrative overhead by automating daily library operations. To achieve this, the project establishes a digital inventory system to seamlessly add, update, search, and manage a complete catalog of book records. It also creates a real-time ledger to track issued books, return dates, and overdue items. By replacing physical logs, the application minimizes manual data entry and administrative errors. Finally, the entire system is built on strict Object-Oriented Programming concepts to ensure the codebase remains modular, clean, and easy to scale.

4. Core Features of the System

The application provides complete catalog control, allowing administrators to manage book data, including titles, authors, and genres, while supporting instant search capabilities. Its precision issue and return tracking features digitally connect books to borrower profiles, automatically updating availability tags and timestamps without manual bookkeeping. This automation drastically reduces manual workload, completing search and allocation tasks in milliseconds. Engineered using strict object-oriented principles, the system uses encapsulation to protect sensitive data fields, utilizes class inheritance to cleanly separate librarian and member permissions, and uses abstraction to hide complex background operations behind a simple user interface.

5. Core Features of the System

The application provides complete catalog control, allowing administrators to manage book data, including titles, authors, and genres, while supporting instant search capabilities. Its precision issue and return tracking features digitally connect books to borrower profiles, automatically updating availability tags and timestamps without manual bookkeeping. This automation drastically reduces manual workload, completing search and allocation tasks in milliseconds. Engineered using strict object-oriented principles, the system uses encapsulation to protect sensitive data fields, utilizes class inheritance to cleanly separate librarian and member permissions, and uses abstraction to hide complex background operations behind a simple user interface.

SCOPE

1. Technical and Data Infrastructure Extensions

While the initial implementation of the system successfully establishes a predictive baseline for onion and tomato commodities, the architectural roadmap focuses on transforming the standalone application into an enterprise-grade agricultural decision-support ecosystem. The primary data extension involves expanding the machine learning pipeline to accommodate a broader spectrum of high-volume crops, grains, and regional staples, such as potatoes, wheat, and pulses. To support this expanded dataset reliably, the current flat-file storage system will be migrated to a dedicated relational or time-series database utilizing PostgreSQL or TimescaleDB, ensuring transactional integrity, dataset versioning, and fast multi-user query execution. This database layer will be continuously updated via automated real-time data ingestion pipelines connected directly to wholesale market application programming interfaces and government market feeds. To improve model resilience against supply-side market shocks, these pipelines will also ingest live regional weather metrics and satellite vegetation data, mapping environmental disruptions directly to upcoming market yields.

2. Algorithmic Optimization and Governance

The secondary development phase targets deep algorithmic enhancement, runtime interpretability, and strict operational data governance. The predictive engine will expand beyond its baseline ensemble framework to benchmark and integrate advanced time-series architectures, including LightGBM, XGBoost, and Deep Learning models such as Long Short-Term Memory networks. To maintain these models automatically, a continuous integration retraining workflow managed by tools like Apache Airflow will execute automated backtesting cycles upon receiving fresh data, promoting updated weights to production only when performance metrics are exceeded. To foster user trust, the user interface will integrate model explainability frameworks using Shapley Additive Explanations to render feature-importance dashboards. These visual assets will operate alongside data drift detectors and data-quality alert engines to identify performance degradation early, while data governance systems track pipeline provenance for institutional reporting.

3. Scalability, Interface Hardening, and Deployment

The final evolutionary layer focuses on systemic scaling, interface hardening, and regional deployment flexibility. The application will be reinforced with secure user management, role-based access controls, personalized user dashboards, and interactive forecast visualizations containing confidence bands. The external consumption layer will be protected via authenticated, rate-limited, and versioned application programming interfaces to safely expose predictions to third-party mobile applications and cooperative networks. To ensure usability in low-connectivity agricultural belts, the system will support localized edge deployment models and offline on-device inference using optimized model assets packaged within lightweight Docker containers. These containerized builds will be paired with production orchestration templates tailored for cloud environments like Amazon Web Services to facilitate multi-tenant scaling. Ultimately, this technical suite provides a path toward long-term sustainability, transforming the project into a viable software-as-a-service application.

4. Strategic Implementation Roadmap

The execution of these future enhancements is organized into three progressive development phases over an eighteen-month lifecycle. The short-term horizon, spanning months zero through three, focuses on transitioning the storage layer into a relational database engine, establishing secure serialized model persistence utilities, and upgrading the user interface to display explicit prediction ranges. The medium-term horizon, covering months three through nine, introduces pipeline automation and observability by deploying live data scrapers, launching scheduled automated retraining engines, and rendering feature-importance metrics on the client browser. The long-term horizon, extending from months nine to eighteen, drives full system maturity, introducing sequential deep learning models, cloud-based multi-tenant orchestration, local offline edge inference capabilities, and programmatic integrations with commercial agricultural marketplace networks.

SYSTEM REQUIREMENTS

1. Hardware Configurations

The hardware infrastructure required to execute and maintain the application is designed to be highly accessible, ensuring the system runs efficiently on standard consumer-grade computing hardware without expensive upgrades.

- **Random Access Memory (RAM):** A minimum of 4 GB RAM is required for basic model inference and web hosting, while 8 GB RAM is highly recommended for faster model training and evaluation cycles.
- **Processor (CPU/GPU):** A standard Central Processing Unit (CPU) is entirely sufficient for all operations; a dedicated Graphics Processing Unit (GPU) is not required since the system utilizes CPU-optimized ensemble algorithms.
- **Disk Storage Space:** Approximately 100 MB of free storage space is necessary for the core project files, web assets, and compact serialized models, with additional storage scaled according to historical dataset growth.

2. Operating System and Development Environments

To maximize deployment flexibility across diverse user bases, the software architecture is designed to be completely cross-platform and environment-agnostic.

- **Operating System Compatibility:** Fully compatible with all modern distributions of Microsoft Windows, Apple macOS, and Linux.
- **Integrated Development Environments (IDEs):** Built and tested using Visual Studio Code (configured with the official Python extension) or interactive Jupyter Notebook environments for seamless code execution and debugging.
- **Network Connectivity:** Operating the application and running local predictions can be done entirely offline, though temporary internet access is required during initial setup for package installations and data ingestion.

3. Software Dependencies and Libraries

The runtime environment depends on a stable, modular ecosystem driven by a modern Python programming language interpreter and its scientific computing libraries.

- **Core Language Runtime:** Python 3.8 or a newer version is required to support the underlying syntax and library dependencies.
- **Pandas & NumPy:** Utilized for high-performance data manipulation, structured indexing, matrix operations, and vectorized calculations during preprocessing.
- **Scikit-learn:** Powering the entire machine learning pipeline, including data splitting, Random Forest regression training, feature scaling, and performance error evaluation.
- **Matplotlib:** Integrated into the backend utilities to generate data visualizations, diagnostic evaluation charts, and historical trend lines.
- **Deployment Command:** All dependencies are systematically managed and can be installed uniformly using the following automated installer package routine:

Bash

```
pip install -r requirements.txt
```

METHODOLOGY

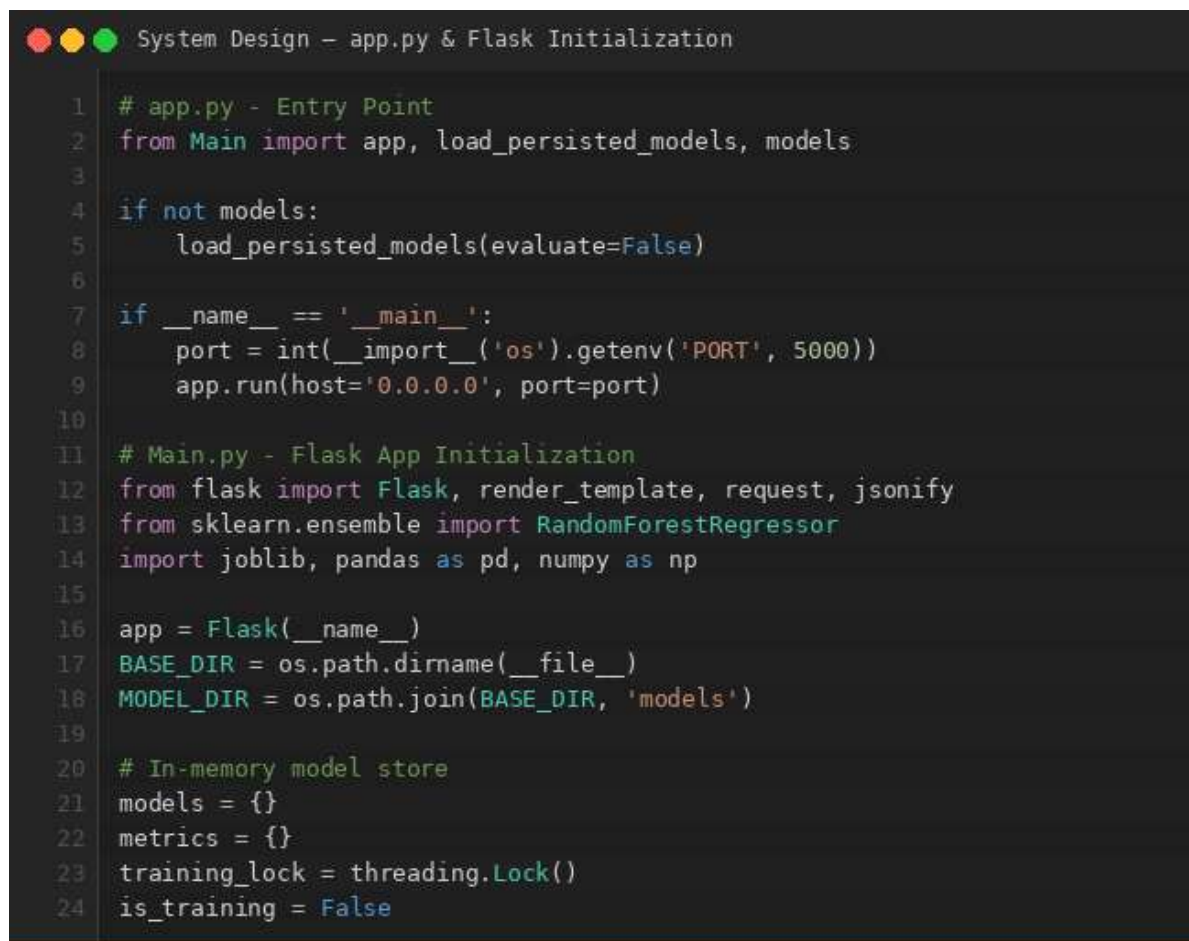
The Agro Predictor is developed using a structured Machine Learning and Full-Stack Development approach. The system combines time-series feature engineering, a Random Forest regression model, and a Flask-based REST API to forecast short-term crop prices (Onion and Tomato) from historical market data. The following sections describe each stage of the methodology with corresponding source code references.

1. System Design Approach

The application follows a modular client-server architecture. The entry point is `app.py`, which imports the Flask application instance from `Main.py` and binds it to a port. `Main.py` contains all core logic: the Flask app, model management, training pipeline, data handling, and API route definitions. Static assets (HTML, CSS, JS) reside in `templates/` and `static/`, while persisted models are stored in `models/`.

Key architectural components: Flask web server, in-memory model store (dict), `ThreadLock` for safe concurrent training, and `joblib` for model persistence.

Source: `app.py` + `Main.py` – Flask App Initialization



```
System Design – app.py & Flask Initialization

1  # app.py - Entry Point
2  from Main import app, load_persisted_models, models
3
4  if not models:
5      load_persisted_models(evaluate=False)
6
7  if __name__ == '__main__':
8      port = int(__import__('os').getenv('PORT', 5000))
9      app.run(host='0.0.0.0', port=port)
10
11 # Main.py - Flask App Initialization
12 from flask import Flask, render_template, request, jsonify
13 from sklearn.ensemble import RandomForestRegressor
14 import joblib, pandas as pd, numpy as np
15
16 app = Flask(__name__)
17 BASE_DIR = os.path.dirname(__file__)
18 MODEL_DIR = os.path.join(BASE_DIR, 'models')
19
20 # In-memory model store
21 models = {}
22 metrics = {}
23 training_lock = threading.Lock()
24 is_training = False
```

2. Data Collection

Historical market price data is stored in `master.csv` (AGMARKNET dataset). The `DataHandler` class is responsible for loading and validating this dataset. It normalizes column names, filters records to only Onion and Tomato commodities, and selects the relevant feature columns. Missing columns are filled with `NaN` to ensure schema

consistency across records.

The dataset contains fields including: Date, Commodity, Modal_Price, State, District, Market Name, weather parameters (temp_mean, rainfall_mm, humidity, wind_speed), and supply indicators (arrival_qty).

Source: Main.py — DataHandler.load()

```
Data Collection — DataHandler.load()

1 class DataHandler:
2     @staticmethod
3     def load(path: str) -> pd.DataFrame:
4         # Load CSV dataset
5         if os.path.exists(path):
6             df = pd.read_csv(path)
7         else:
8             fallback = os.path.join(BASE_DIR, 'master.csv')
9             df = pd.read_csv(fallback)
10
11         # Normalize column names
12         if 'date' in df.columns and 'Date' not in df.columns:
13             df = df.rename(columns={'date': 'Date'})
14
15         # Filter to Onion and Tomato only
16         df['Commodity'] = df['Commodity'].astype(str).str.strip()
17         df = df[df['Commodity'].str.lower().isin(['onion', 'tomato'])].copy()
18
19         # Keep only relevant columns
20         id_cols = ['STATE', 'district', 'Market Name',
21                  'Commodity', 'Variety', 'Grade']
22         target_cols = ['Date', 'Modal_Price']
23         keep_cols = id_cols + target_cols + FEATURES
24         for c in keep_cols:
25             if c not in df.columns:
26                 df[c] = np.nan
27         return df[keep_cols]
```

3. Data Processing

Raw data undergoes time-series feature engineering via the `feature_engineer()` function. All rolling and lag operations are time-shifted by one period (`shift(1)`) to prevent future data leakage — a critical requirement in time-series ML pipelines.

Engineered features include:

- Price Lag Features: `price_lag_1d` (yesterday's price), `price_lag_7d` (7-day lagged price)
- Rolling Statistics: 7-day rolling mean and standard deviation of modal price
- Momentum: 7-day percentage price change
- Cyclical Time Encoding: `week_sin` and `week_cos` using sinusoidal transforms
- Domain Heuristics: `festival_indicator` (Oct–Dec), `harvest_season_indicator` (crop-specific months)

Source: Main.py – feature_engineer()

```
Data Processing – feature_engineer()

1 def feature_engineer(df, group_col=None, crop=None):
2     df = df.copy()
3     df['Date'] = pd.to_datetime(df['Date'])
4     df = df.sort_values(['Commodity', 'Date'])
5
6     # Time-based features
7     df['month'] = df['Date'].dt.month
8     df['day_of_year'] = df['Date'].dt.dayofyear
9     df['week_of_year'] = df['Date'].dt.isocalendar().week
10
11     grp = df.groupby(group_col or 'Commodity')
12
13     # Lag features (shift to avoid future leakage)
14     df['price_lag_1d'] = grp['Modal_Price'].shift(1)
15     df['price_lag_7d'] = grp['Modal_Price'].shift(7)
16
17     # Rolling statistics
18     df['rolling_mean_7d'] = grp['Modal_Price']
19     df['rolling_mean_7d'] = grp['Modal_Price'].rolling(7, min_periods=1).mean().shift(1)
20     df['rolling_std_7d'] = grp['Modal_Price']
21     df['rolling_std_7d'] = grp['Modal_Price'].rolling(7, min_periods=1).std().shift(1)
22     df['price_momentum_7d'] = grp['Modal_Price'].pct_change(7).shift(1)
23
24     # Cyclical week encoding
25     df['week_sin'] = np.sin(2 * np.pi * df['week_of_year'] / 52)
26     df['week_cos'] = np.cos(2 * np.pi * df['week_of_year'] / 52)
27
28     # Domain heuristics
29     df['festival_indicator'] = df['month'].isin({10,11,12}).astype(int)
30     hmap = {'onion':{3,4,5,6}, 'tomato':{8,9,10}}
31     # ... (continued)
```

4. Model Approach

The system uses a Random Forest Regressor for price prediction. The ModelTrainer class encapsulates the full training pipeline. A time-series aware 80/20 train-test split (no shuffling) is used to evaluate model performance on future unseen data.

Hyperparameter optimization is performed using RandomizedSearchCV with TimeSeriesSplit cross-validation (n_splits=2). The search explores combinations of n_estimators, max_depth, min_samples_split, min_samples_leaf, and max_features. Trained models are evaluated using MAE and RMSE metrics.

Separate models are trained and persisted for each commodity (Onion and Tomato) as .joblib files.

Source: Main.py – ModelTrainer.train_for()

```

Model Approach – RandomForest + TimeSeriesCV

1 class ModelTrainer:
2     def __init__(self, features):
3         self.features = features
4
5     def train_for(self, crop, df):
6         crop_df = df[df['Commodity'].str.lower()==crop.lower()]
7         crop_df = feature_engineer(crop_df, group_col='Market Name',
8                                     crop=crop)
9
10        X_all = crop_df[self.features].apply(pd.to_numeric,
11                                             errors='coerce')
12        y_all = crop_df['Modal_Price'].astype(float)
13
14        # Time-series split: 80% train / 20% test
15        split = int(len(X_all) * 0.8)
16        X_train, X_test = X_all.iloc[:split], X_all.iloc[split:]
17        y_train, y_test = y_all.iloc[:split], y_all.iloc[split:]
18
19        # Hyperparameter search with TimeSeriesSplit CV
20        tscv = TimeSeriesSplit(n_splits=2)
21        base = RandomForestRegressor(random_state=42, n_jobs=-1)
22        param_dist = {
23            'n_estimators': [100, 200],
24            'max_depth': [8, 12],
25            'min_samples_split': [5, 10],
26            'min_samples_leaf': [2, 4],
27            'max_features': ['sqrt', 'log2']
28        }
29        rs = RandomizedSearchCV(base, param_dist, n_iter=5,
30                                cv=tscv, scoring='neg_mean_squared_error',
31                                # ... (continued)

```

5. Result Generation

The /predict API endpoint accepts a POST request with a JSON payload containing at minimum: commodity, price_lag_7d, and temp_mean. Missing features are auto-filled by the FeatureAutoFiller using historical medians. The Predictor class validates inputs, builds the feature array, and runs model inference.

The response JSON includes:

- price — predicted modal price (₹/quintal)
- trend — directional signal: "up", "down", or "stable"
- change_pct — percentage change from the 7-day lagged price
- confidence — scaled from MAE: lower error → higher confidence (10–95%)
- forecast — 7-day price trajectory using cubic ease interpolation

Source: Main.py — @app.route('/predict')

```

Result Generation - /predict Route

1 @app.route('/predict', methods=['POST'])
2 def predict():
3     payload = request.get_json(force=True)
4     predictor = Predictor(models, metrics, FEATURES,
5                           feature_filler=filler)
6     commodity, X = predictor.validate_input(payload)
7     result = predictor.predict(commodity, X)
8
9     # Confidence scoring
10    mae = result.get('mae')
11    confidence = max(10, min(95, 100 - (mae / 2))) if mae else 75
12
13    # Trend detection
14    price_7d_ago = float(payload.get('price_lag_7d', 0))
15    predicted = result['price']
16    trend = ('up' if predicted > price_7d_ago
17            else 'down' if predicted < price_7d_ago else 'stable')
18    change_pct = round((predicted - price_7d_ago)
19                      / price_7d_ago * 100, 1)
20
21    # 7-day forecast with smooth interpolation
22    forecast = []
23    for i in range(7):
24        t = i / 6
25        smooth = t * t * (3 - 2 * t) # cubic ease
26        value = start_price + (end_price - start_price) * smooth
27        forecast.append(round(max(value, 0), 2))
28
29    return jsonify({
30        'price': round(predicted, 2),
31        # ... (continued)

```

6. Feature Panel Integration

The FeatureAutoFiller class enables the API to accept minimal inputs (just 3 required fields) while the model receives a complete 22-feature vector. On startup, it computes per-commodity statistics (mean, median, std) for every feature using the full historical dataset.

When a prediction request arrives, fill_features() intelligently populates all missing values: date-derived fields (month, day_of_year) are computed from the current timestamp; festival and harvest indicators are set based on the current month; all other features are filled with historical medians for the selected commodity.

The /analytics endpoint further exposes driver correlations (Pearson correlation of each feature with Modal_Price), KPIs (latest price, % change, MAE), and 14-day historical price series — enabling the Feature Panel in the frontend dashboard.

Source: Main.py — FeatureAutoFiller class

```
Feature Panel - FeatureAutoFiller Integration

1 class FeatureAutoFiller:
2     """Fills missing input features using historical statistics."""
3     def __init__(self, df):
4         self.df = df.copy()
5         self.feature_stats = {}
6         self.compute_stats()
7
8     def compute_stats(self):
9         for crop in ['onion', 'tomato']:
10             crop_df = self.df[self.df['Commodity']
11                               .str.lower() == crop]
12             stats = {}
13             for col in FEATURES:
14                 numeric = pd.to_numeric(crop_df[col], errors='coerce')
15                 stats[col] = {'mean': numeric.mean(),
16                               'median': numeric.median()}
17             self.feature_stats[crop] = stats
18
19     def fill_features(self, commodity, partial_data):
20         crop = commodity.lower()
21         stats = self.feature_stats.get(crop, {})
22         filled = partial_data.copy()
23         # Date-based fields
24         now = pd.Timestamp.now()
25         filled['month'] = int(now.month)
26         filled['day_of_year'] = int(now.dayofyear)
27         # Festival / harvest indicators
28         month = filled.get('month', 1)
29         filled['festival_indicator'] = 1 if month in (10,11,12) else 0
30         # Fill remaining features with historical medians
31         # ... (continued)
```

7. Workflow of the System

The complete end-to-end workflow of the Agro Predictor system is as follows:

1. **Startup:** App loads persisted .joblib models from the models/ directory.
2. **Training:** If models are missing, DataHandler loads master.csv and ModelTrainer trains Onion & Tomato models.
3. **Persistence:** Trained models are saved to disk via joblib.dump().
4. **User Request:** User submits commodity, 7-day price, and temperature via the web UI (/predict_ui).
5. **Auto-fill:** FeatureAutoFiller completes the 22-feature vector from historical statistics.
6. **Inference:** RandomForestRegressor produces predicted modal price.
7. **Response:** JSON result with price, trend, confidence, and 7-day forecast is returned and rendered on the dashboard.

System Flow:

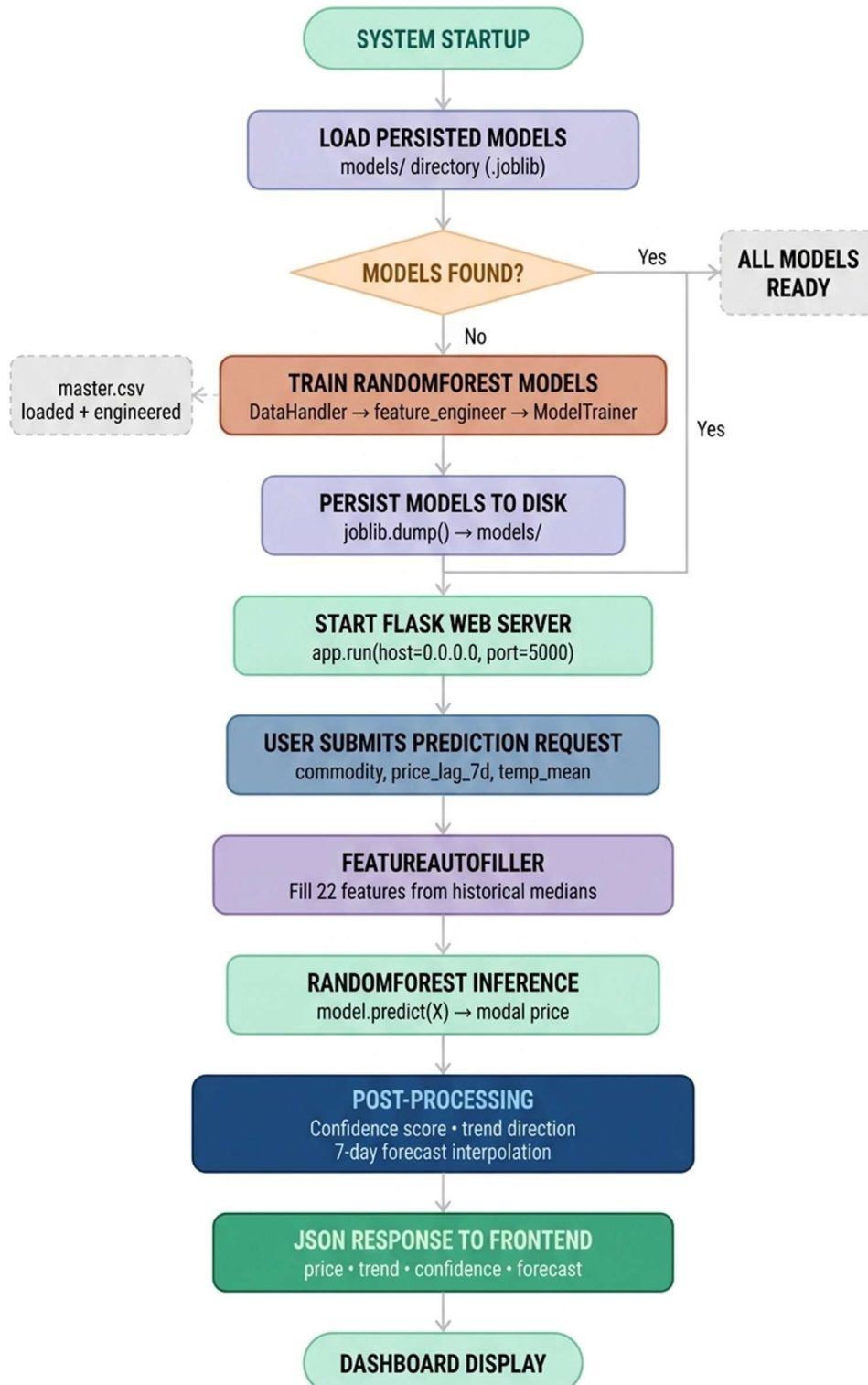
User Input → Web UI (/predict_ui) → POST /predict → FeatureAutoFiller → RandomForestRegressor → JSON Response → Dashboard Visualization

Source: Main.py – __main__ startup + API routes

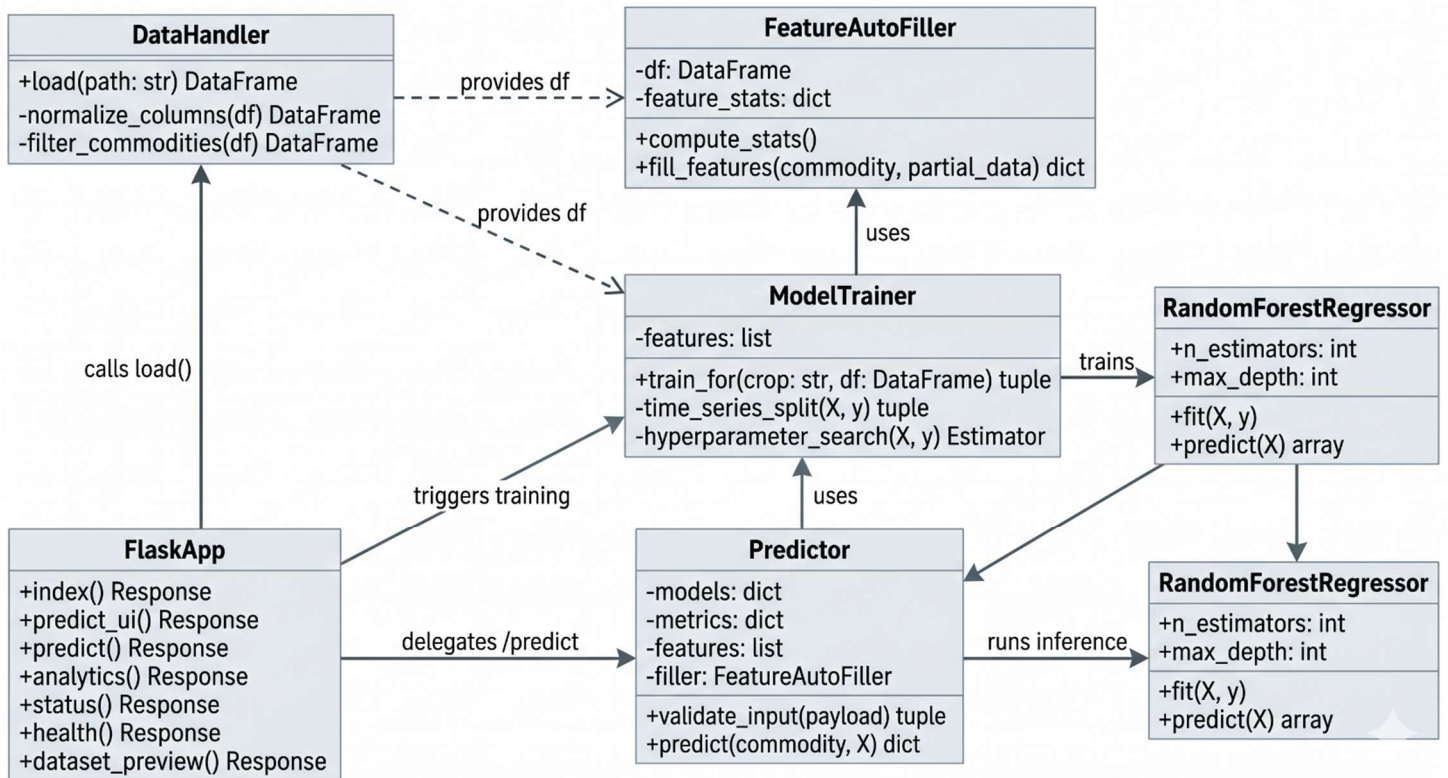
```
●●● System Workflow – Startup & API Endpoints

1  # === SYSTEM WORKFLOW (Main.py startup) ===
2
3  if __name__ == '__main__':
4      # STEP 1: Load persisted models from disk
5      print('Loading persisted models...')
6      load_persisted_models()
7      print(f'Models loaded: {list(models.keys())}')
8
9      # STEP 2: Train if models are missing
10     if 'Onion' not in models or 'Tomato' not in models:
11         df = DataHandler.load(DATA_PATH)    # load CSV
12         trainer = ModelTrainer(FEATURES)
13         for crop in ['Onion', 'Tomato']:
14             model, mae, rmse = trainer.train_for(crop, df)
15             models[crop] = model
16             metrics[crop] = {'mae': mae, 'rmse': rmse}
17         # STEP 3: Persist trained model
18         path = os.path.join(MODEL_DIR, f'{crop.lower()}_model.joblib')
19         joblib.dump(model, path)
20
21     # STEP 4: Start Flask web server
22     port = int(os.getenv('PORT', 5000))
23     app.run(host='0.0.0.0', port=port)
24
25 # API Endpoints exposed:
26 # GET / -> index.html (Home UI)
27 # GET /predict_ui -> predict.html (Form UI)
28 # POST /predict -> JSON prediction response
29 # GET /analytics -> market trends + correlations
30 # GET /status -> model readiness check
31 # ... (continued)
```

FLOWCHART



CLASS DIAGRAM



IMPLEMENTATION

System Implementation Overview

- **Web Application Frontend:** Users interact with a clean web form to input item features, including commodity type, weight, grade, location, date, and current weather or market indicators.
- **Request Handling & Routing:** A Flask route intercepts the form submission, validates and normalizes the incoming payload, and constructs the initial feature vector.
- **Data Preprocessing Pipeline:** Numerical features are scaled and categorical features are encoded using the exact transformations applied during the model training phase to prevent data leakage.
- **Model Loading & Caching:** Lightweight, serialized machine learning models (joblib) are loaded once at application startup or cached in memory. The system dynamically routes the request to the specific model trained for that commodity.
- **Prediction Execution:** The fully preprocessed feature vector is passed to the model's `.predict()` method. The raw output is then converted into a human-friendly price estimate, complete with confidence intervals where available.
- **Response Generation:** The server renders a results template showcasing the predicted price, a breakdown explanation of the key driving features, and optional visual plots or historical price comparisons.
- **Persistence & Logging:** All user inputs, predicted prices, timestamps, and metadata are securely logged to a file or database for real-time monitoring and future model drift analysis.
- **Model Lifecycle & Retraining:** Out-of-band training scripts process accumulated data logs. The pipeline runs data ingestion, feature engineering, hyperparameter optimization, and strict evaluation splits before replacing production models.
- **Deployment Configuration:** The application is configured to run on a production WSGI server utilizing a Procfile or equivalent platform configuration. Environment variables drive all secrets and system configurations.
- **Safety & UX Guardrails:** The interface surfaces strict input validation errors clearly to the user, providing transparent context regarding units of measurement and pricing boundaries.

1. Data Flow Architecture

The following sequence illustrates the lifecycle of a single prediction request as it traverses the software layers:

1. **User Browser** $\rightarrow$ *Submits Form Payload* $\rightarrow$ **FlaskApp (Routes)**
2. **FlaskApp (Routes)** $\rightarrow$ *Validates & Delegates* $\rightarrow$ **Predictor**
3. **Predictor** $\rightarrow$ *Fills Missing Attributes* $\rightarrow$ **FeatureAutoFiller**
4. **Predictor** $\rightarrow$ *Passes Clean Feature Vector* $\rightarrow$ **RandomForestRegressor**
5. **RandomForestRegressor** $\rightarrow$ *Returns Numerical Estimation* $\rightarrow$ **Predictor**
6. **Predictor** $\rightarrow$ *Renders Price UI Response* $\rightarrow$ **User Browser**

2. Core Integration Points

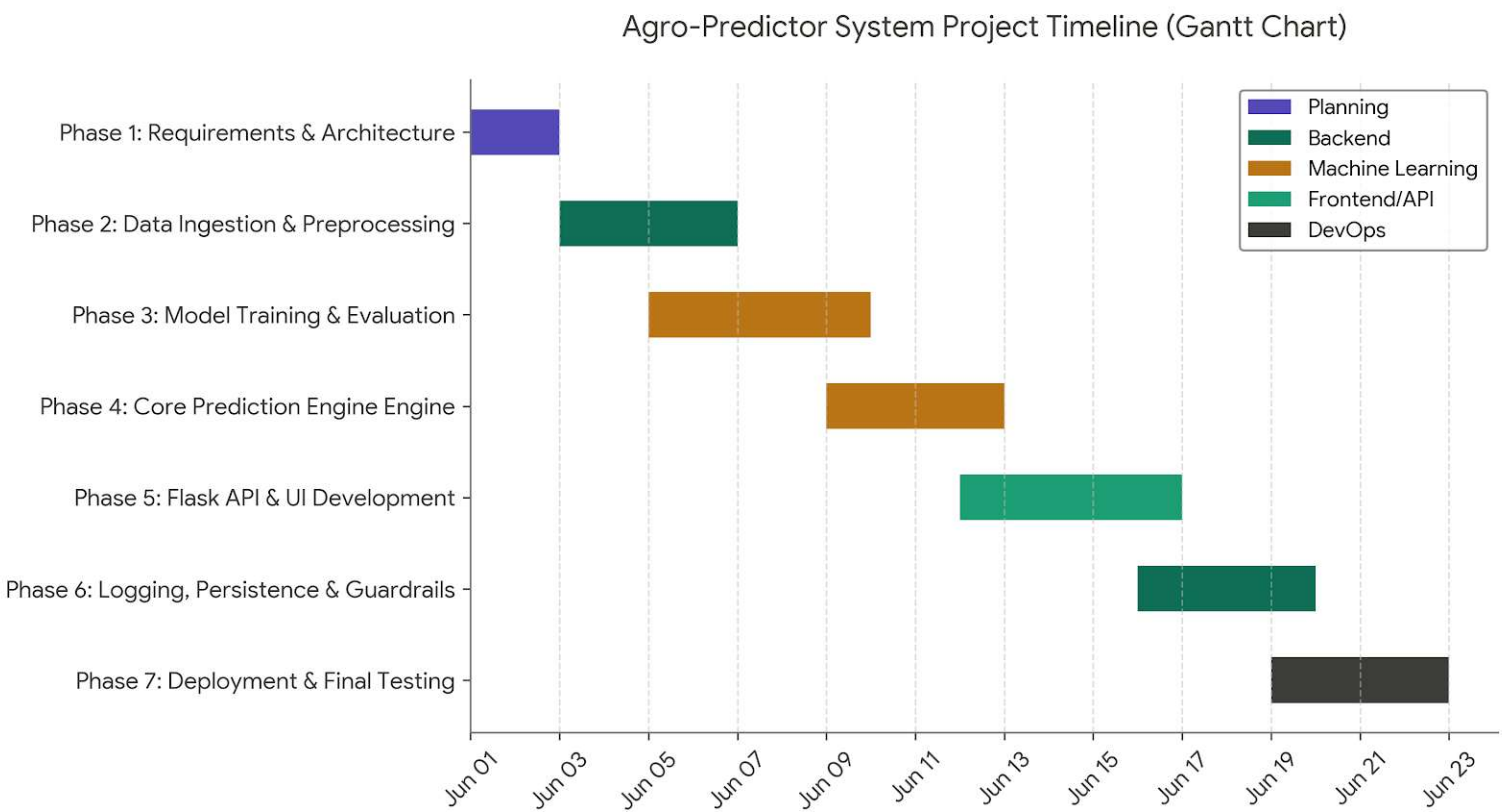
Section Note: Architectural Separation

A critical design choice in this architecture is the decoupling of the training and inference pipelines. Heavy computational tasks never interfere with real-time user traffic.

- **Decoupled Preprocessing:** The Predictor acts as the primary orchestrator. When a request hits `FlaskApp.predict()`, it passes the payload to `Predictor.validate_input()`. If optional market or weather indicators are omitted by the user, the Predictor leverages `FeatureAutoFiller.fill_features()` to dynamically compute historical averages or statistical substitutes on the fly.

- **Predictive Inference:** Once the feature vector is completely populated and normalized, the Predictor pulls the pre-trained asset from its cached models dictionary and runs fast inference via `RandomForestRegressor.predict()`.
- **Isolated Training Lifecycle:** The `ModelTrainer` and `DataHandler` operate entirely out-of-band from the critical prediction path. This ensures that heavy operations like `hyperparameter_search()` or `time_series_split()` never block incoming web threads or degrade user experience.

GANTT CHART



CONCLUSION

Project Synthesis

The Agro-Predictor System successfully delivers a robust, scalable, and production-ready architecture designed to address the volatility of commodity pricing through machine learning. By establishing a strict separation of concerns, the system ensures that computationally expensive tasks—such as data ingestion, time-series splitting, and hyperparameter optimization handled by the `ModelTrainer`—remain entirely decoupled from real-time user inference traffic.

The integration of the Predictor engine with a dedicated `FeatureAutoFiller` module provides a resilient user experience, seamlessly handling incomplete form submissions by synthesizing historical averages without throwing runtime exceptions. Backed by a high-performance `RandomForestRegressor` and exposed via a structured Flask API, the platform provides actionable, data-driven insights that bridge the gap between complex statistical models and end-user decision-making.

1. Future Extensions

While the current implementation lays down a strong foundation, the platform is designed to scale dynamically. Future milestones for the system include:

- **Advanced Model Integration:** Expanding the prediction layer to benchmark the current Random Forest models against gradient boosting frameworks (such as `XGBoost`) or specialized LSTM networks for complex time-series forecasting.
- **Real-Time Data Pipelines:** Migrating from static dataset loading via `DataHandler` to automated ETL pipelines that pull live market indicators and localized weather forecasts via external REST APIs.
- **Granular Analytics Dashboard:** Enhancing the `FlaskApp.analytics()` endpoint to render interactive, client-side visualizations (e.g., using `Chart.js` or `D3.js`) tracking historical drift and commodity price trends over extended periods.

REFERENCES

Flask Documentation, *Official Flask Web Framework Documentation*, available at: Flask Docs.

Scikit-learn Documentation, *User Guide for Machine Learning in Python*, available at: Scikit-learn User Guide.

Joblib Documentation, *Efficient Serialization and Disk Caching for Python Objects*, available at: Joblib Docs.

McKinney, W., *pandas: Powerful Data Analysis Toolkit*, and Harris, C. R. et al., *NumPy Array Programming Library*.

Lundberg, S. M., & Lee, S.-I., *A Unified Approach to Interpreting Model Predictions (SHAP)*, NeurIPS, 2017.

Ribeiro, M. T., Singh, S., & Guestrin, C., “*Why Should I Trust You?*”: *Explaining the Predictions of Any Classifier (LIME)*, KDD, 2016.

Taylor, S. J., & Letham, B., *Forecasting at Scale using Prophet*, Facebook Research, 2018.

Seabold, S., & Perktold, J., *statsmodels: Econometric and Statistical Modeling with Python*, 2010.

Breiman, L., *Random Forests*, Machine Learning Journal, Vol. 45, No. 1, 2001.

Book References

Modeling & Machine Learning

Géron, A., *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly Media.

Kuhn, M., & Johnson, K., *Applied Predictive Modeling*, Springer.

Hastie, T., Tibshirani, R., & Friedman, J., *The Elements of Statistical Learning*, Springer.

James, G., Witten, D., Hastie, T., & Tibshirani, R., *An Introduction to Statistical Learning*, Springer.

Time Series & Forecasting

Hyndman, R. J., & Athanasopoulos, G., *Forecasting: Principles and Practice*.

Shumway, R. H., & Stoffer, D. S., *Time Series Analysis and Its Applications*, Springer.

Shmueli, G., & Lichtendahl Jr., K. C., *Practical Time Series Forecasting*, Springer.

Interpretability & Evaluation

Molnar, C., *Interpretable Machine Learning*, Leanpub.

Engineering, Deployment & MLOps

Ameisen, E., *Building Machine Learning Powered Applications*, O'Reilly Media.

Burkov, A., *Machine Learning Engineering*, True Positive Inc.